

Exact per-position early exit: three inference-time upgrades to the FLEX-UF ladder

Supplementary experiment report — the main experiment (`runs/RECIPE512`, pinned at epoch 4) is not modified anywhere in this document.

Abstract. FLEX-UF saves decoder compute by letting each 256 px tile of a frame leave a shared trunk at its own depth, and reports 27.64% of a released DCVC-UF decode saved at a 0.1 dB budget over 53 CTC frames. This report shows that three properties of that system are artefacts of the cut rather than of the ladder, and that removing them is worth 8.38 points of saving with no weight retrained. First, tiling pays a seam out of the distortion budget: the deployed operating point spends about 0.015 of its 0.1 dB on the cut itself. Second, replacing tiles with per-position depth, computed where a max-plus dilation of the allocated depth says a block is needed, is exact — each position receives what a full-frame decode to its own depth would have produced — and verified to 0.0002 dB on the full test set. Third, once there is no cut, the split depth that existed to control the seam has no purpose, and the two shallow rungs it locks out turn out to be already trained. Integrated in the Bjontegaard style over a common interval, the frontier moves from 26.81% to 31.05% BD-saving and from 0.0618 to 0.0334 dB BD-quality. A trained decoder-side router with no signalled map reaches 27.79%, above the tiled oracle. We also report what did not work.

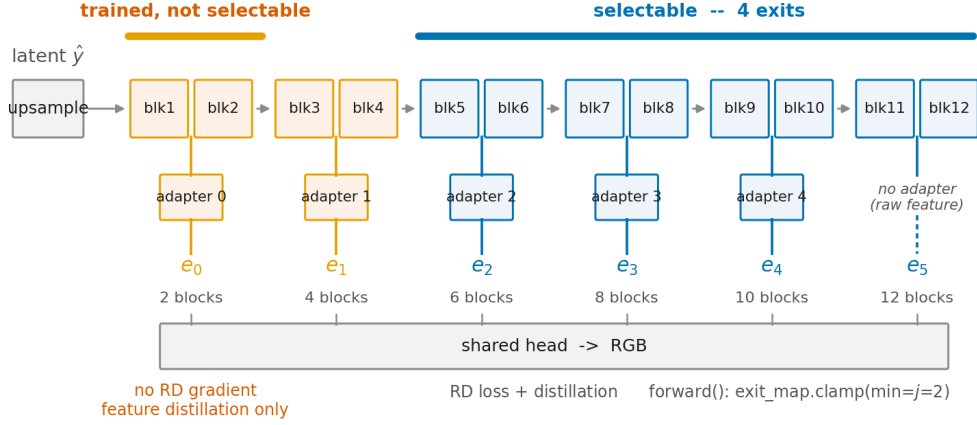
1 Introduction

An early-exit decoder spends less arithmetic on the parts of an image that do not need it. FLEX-UF does this by cutting the frame into tiles and giving each tile its own exit from a twelve-block trunk. The cut is what makes the idea implementable: a tile is a contiguous region, so a tile that leaves early simply stops, and the compute saved is a rectangle.

The cut is also what the idea pays for, in three ways that are easy to miss because each of them looks like a property of the ladder. (i) A convolution at a tile border reads a padded value instead of its neighbour, and the resulting error is charged to the distortion budget the system is trying to respect. (ii) The finer the tiles, the more border per unit area, so the allocation cannot be made finer than the seam allows — 256 px is not a modelling choice but a seam budget. (iii) Because the seam is worst where the features are least settled, the first blocks of the trunk are made shared, which removes the shallowest rungs of the ladder from the deployed decoder entirely.

This report removes the cut and measures what each of the three was costing. Nothing is retrained: every number below is produced by the pinned epoch-4 checkpoint of the main experiment, on the same 53 CTC frames, against the same released DCVC-UF decoder, at the same 0.1 dB budget in the same per-frame decibel.

2 The ladder as trained and as deployed



Sekil 1. The main experiment's ladder. Six exits are trained; the deployed forward can select four. `forward()` applies `exit_map.clamp(min=j)` with $j=2$, so a tile assigned exit 0 or 1 decodes as exit 2. The training loss does not know about the clamp: the exit weights in `flexuf/losses.py` are `w[:K-1] = aux_weight`, with no reference to the split. Under `--train_patched` the RD term is computed through `forward_random_depth`, which clamps, so adapters 0 and 1 receive a gradient only from the feature-space distillation term. The deepest exit wears no adapter at all, which is what makes it reproduce stock DCVC-UF.

Two facts in this figure carry the rest of the report. The first is that the ladder has six rungs and the decoder uses four. The second is that the two unused rungs are nevertheless trained — not as well as they could be, since no pixel loss ever reaches them, but trained: their adapters carry real weights in the checkpoint and their reconstructions improve monotonically with epoch.

3 The seam is paid out of the distortion budget

The seam is usually discussed as an artefact one can see. It is more useful to price it, because the system already pays for it in the one currency it is constrained by.

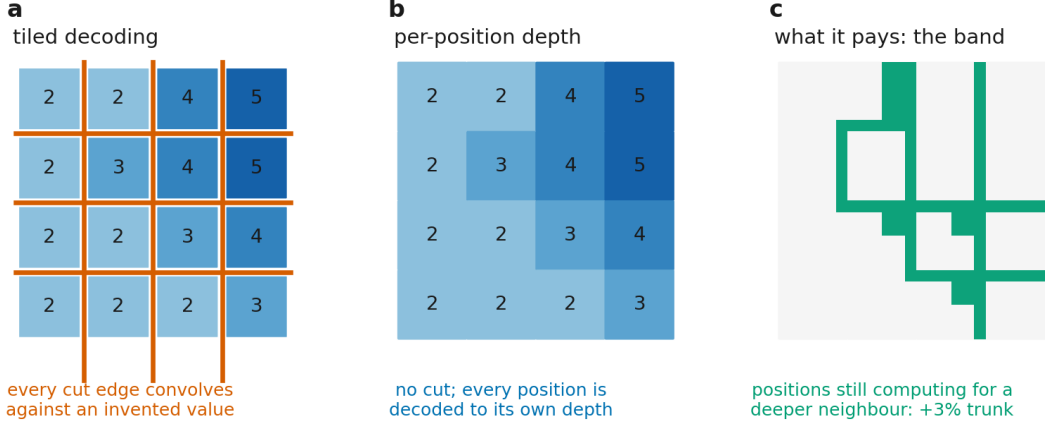
The evidence is threefold and the three measurements are independent. **(i)** The deployed operating point is found by a two-level bisection in `scripts/raterank_curve.py`: an inner bisection on a cheap per-tile table, then a correction against a real decode of the resulting mixed-depth map. That correction pulls the inner target from 0.100 dB to about 0.085 — roughly a sixth of the budget is spent on the fact that neighbouring tiles have different depths. **(ii)** Decoding one routed map two ways on the same weights, the tiled decode reaches 0.0988 dB and the per-position decode 0.0864 dB, for 0.6% more trunk. **(iii)** The frontier floors differ by the same amount: with every cell at the deepest exit the tiled decoder still sits 0.022 to 0.048 dB above the release, the per-position one 0.001 to 0.020.

4 Per-position depth with a dilated receptive field

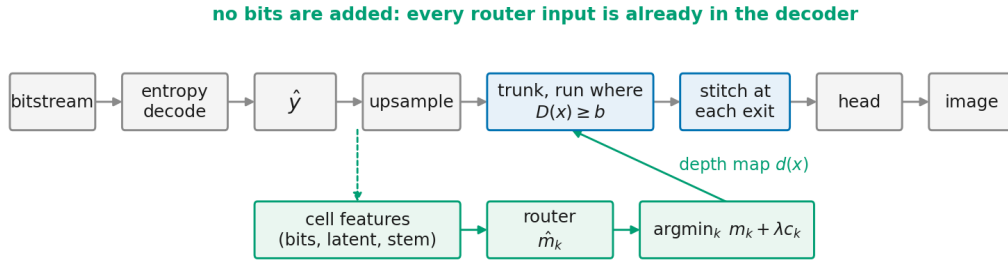
Let $d(x)$ be the depth allocated at position x , in trunk blocks. A block b must be computed at x whenever some position y that needs at least b blocks lies within $b - c$ of it, which is the max-plus dilation

$$D(x) = \max_y [d(y) - \text{dist}(x, y)]$$

and the decode that runs block b exactly where $D \geq b$ gives every position what a full-frame decode to its own depth would have given it. There is no cut, so there are no invented values; a position that stops early keeps being computed only for as long as a deeper neighbour's receptive field still reaches it. The cost is a band whose width is the depth *difference* between neighbours rather than the whole tile.



Sekil 2. (a) Tiled decoding cuts, and every cut edge convolves against a padded value. (b) Per-position depth does not cut. (c) What it pays instead: the positions that keep computing to serve a deeper neighbour. The band in (c) is computed by dilating the map in (a)–(b), not drawn by hand, which is why it is thin and irregular rather than a fixed halo.



Sekil 3. The decode with the allocation path drawn in. The router reads quantities the decoder already has — per-cell bits from the entropy model, latent statistics, and the stem feature — so it adds nothing to the bitstream and its own cost is 0.009% of a decode. The map it produces is dilated once, and the trunk then runs where $D(x) \geq b$.

4.1 Exactness, verified rather than argued

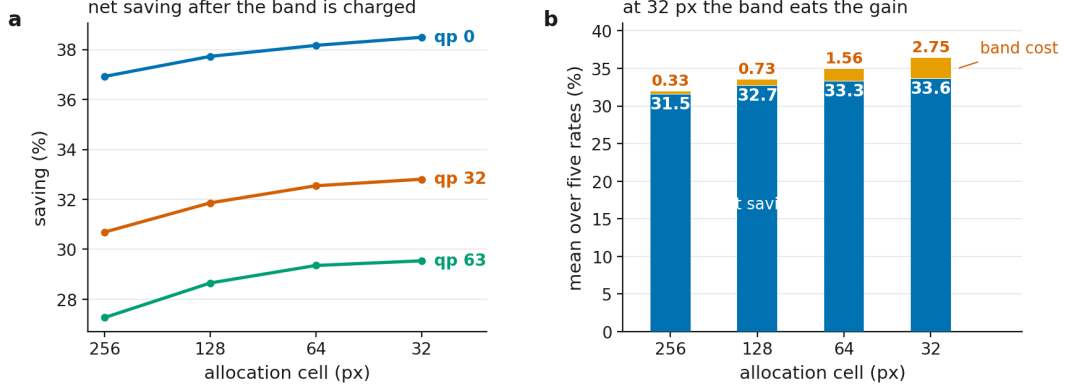
The argument above covers the trunk. It does not cover the head, which has its own 3×3 and at a depth boundary reads one cell across into a feature that left the trunk at a different depth. We therefore compare the idealised decode — each position read from a full-frame decode at its own depth — against a real per-position decode that stitches the exit features and runs the head once, on all 53 frames:

Tablo 1. The head term, measured on the full test set at 64 px cells. It is below 0.0002 dB and negative, so the sweeps in this report are if anything conservative.

rate	assembled (dB)	real decode (dB)	head term (dB)
qp 0	+0.0919	+0.0918	-0.0001
qp 32	+0.0967	+0.0965	-0.0002
qp 63	+0.0974	+0.0973	-0.0001

5 Granularity stops costing distortion

With tiles, halving the tile side doubles the border per unit area and the seam grows with it; 256 px is the size at which that is affordable. With per-position depth there is no border at any granularity, so a finer allocation costs only compute — the band — and the trade can be measured instead of guessed.



Sekil 4. Allocation cell against saving, at the 0.1 dB budget over 53 CTC frames. (a) Net saving per rate. (b) The same, averaged over five rates, with the dilation band charged on top: the band grows from 0.33 to 2.75 points and at 32 px it consumes the remaining gain, which is why 64 px is the operating choice.

Table 2. Per-position decoding at four allocation granularities, oracle allocation, saving against a released decode. The band column is already subtracted from the savings.

cell	q0	q16	q32	q48	q63	mean	band
256 px	36.92	33.99	30.69	28.69	27.26	31.51	0.33
128 px	37.73	34.98	31.86	30.11	28.65	32.66	0.73
64 px	38.17	35.56	32.54	30.76	29.35	33.27	1.56
32 px	38.49	36.09	32.81	31.02	29.54	33.59	2.75

A finer map also costs bits when it is signalled. The main experiment transmits 64 to 91 bits per frame at 256 px; sixteen times as many cells is still under 0.0005 bpp against a working range of 0.25 to 0.64, and the parameter-free decoder-side router transmits nothing at all.

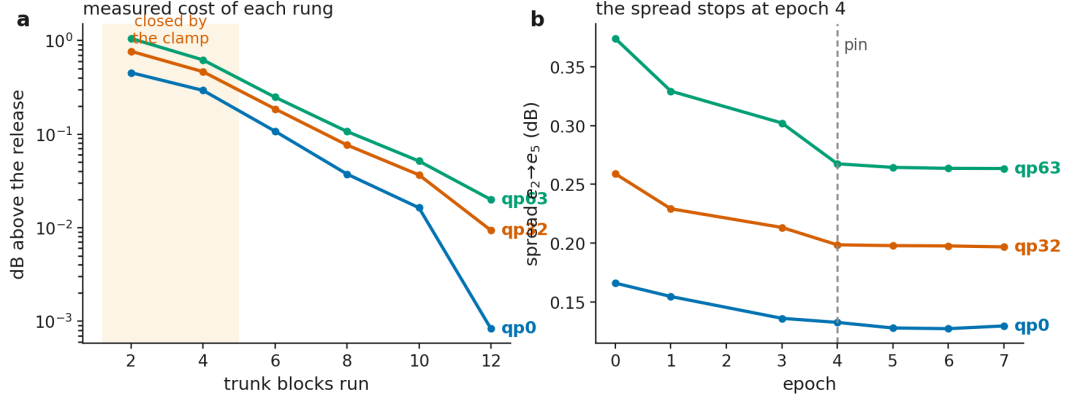
6 The split depth is an artefact of the cut

The split exists because a cut made before the features have settled produces the worst seams. It is therefore a property of tiling, not of the ladder, and once there is no cut it has no job. What it costs is that `forward()` clamps the exit map to $j=2$, so the two shallowest rungs cannot be selected.

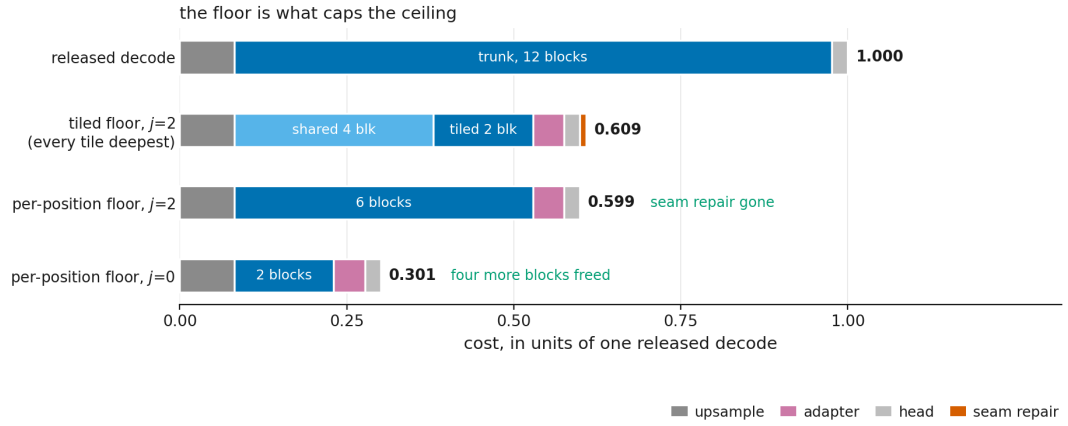
Those rungs are trained. Their adapters carry real weights in the checkpoint, and their errors on the test set are ordinary continuations of the ladder rather than degenerate:

Table 3. Every rung's error on the 53 CTC frames, in dB above the released decoder, measured with `forward_all_exits`, which does not clamp. The first two columns are the rungs the deployed decoder cannot reach.

rate	e0 (2 blk)	e1 (4 blk)	e2 (6 blk)	e3 (8 blk)	e4 (10 blk)	e5 (12 blk)
qp 0	+0.4540	+0.2931	+0.1073	+0.0373	+0.0163	+0.0008
qp 32	+0.7678	+0.4644	+0.1859	+0.0764	+0.0366	+0.0093
qp 63	+1.0533	+0.6205	+0.2480	+0.1066	+0.0514	+0.0199



Sekil 5. (a) The measured cost of each rung, per rate, on a log axis; the shaded region is what the clamp removes. (b) The spread between the shallowest selectable rung and the deepest, by epoch: it stops improving at the pinned epoch, which is the same conclusion the saving series and the training log reach independently.



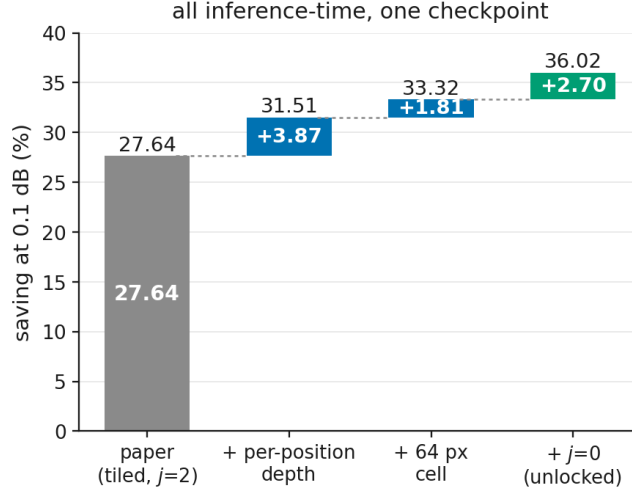
Sekil 6. Where one decode's compute goes. The floor is the cost with every cell at the shallowest exit it may take, and it is what caps the ceiling: 0.609 for the tiled ladder, 0.599 once per-position decoding removes seam repair, and 0.301 once the clamp stops forcing four shared blocks on every position. The upsample, adapter and head terms are irreducible; the trunk is not.

Unlocking them is a change to one line of the decode path. With per-position decoding at 64 px cells, the oracle allocation at the 0.1 dB budget moves as follows.

Table 4. Oracle saving at 0.1 dB, per-position decoding, 64 px cells. The gain is concentrated at low rate, where the two-block rung sits only 0.45 dB above the release and 37% of cells can afford it; at the highest rate it is zero.

split	q0	q16	q32	q48	q63	mean
$j=2$ (as deployed)	38.63	35.75	32.57	30.53	29.14	33.32
$j=0$ (unlocked)	45.56	40.16	33.84	31.46	29.09	36.02

7 Results

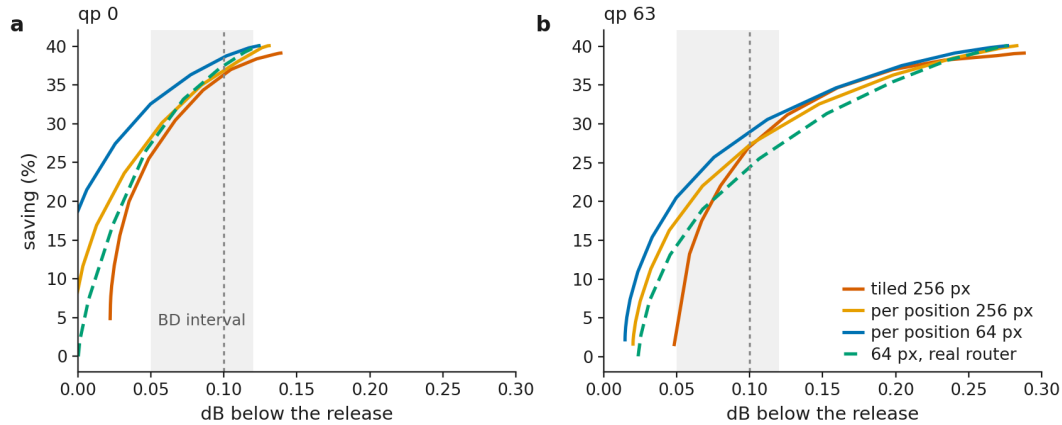


Sekil 7. The three levers, applied in order to the pinned epoch-4 checkpoint. None of them retrains a weight; each is a change to how the existing weights are run.

Reading a frontier at one budget is fragile, and this project has been bitten by it before, so the headline is integrated. `scripts/bd_saving.py` takes its lower limit from each curve's own floor; since the floors differ by exactly the seam, integrating each from its own floor would compare two different intervals and call it one number. All rows below are integrated over the common interval 0.05–0.12 dB, in the per-frame decibel, with saving measured against a released decode.

Tablo 5. Bjontegaard integration over a common interval. The last row uses a decoder-side router with no signalled map and no retrained weight, and still exceeds the tiled oracle on both measures.

decoder	BD-saving (%)	BD-quality (dB)
tiled 256 px, oracle — the paper today	26.81	0.0618
per position 256 px, oracle	28.78	0.0469
per position 64 px, oracle	31.05	0.0334
per position 64 px, trained router	27.79	0.0544

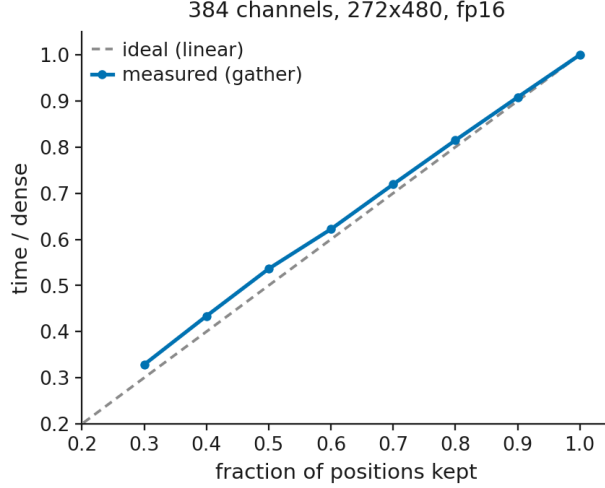


Sekil 8. The compute–quality frontier at the extreme rates. The shaded band is the interval the BD numbers integrate; the dashed line is the 0.1 dB operating point. The tiled curve is rebased onto the same axes as the others — saving against a released decode and the per-frame decibel — because `paper_curve.py` writes the other convention in each pair, and on the raw axes the tiled curve appears above the per-position ones at qp63.

8 Does it run faster, or only count fewer MACs?

Every saving in this project is counted in multiply-accumulates, which is the right unit for arithmetic and the wrong one for time. A mask that skips 40% of positions saves nothing if a dense 3×3 still sweeps the plane, and the adaptive-computation literature is candid that pixel-wise sparse convolution is not hardware friendly.

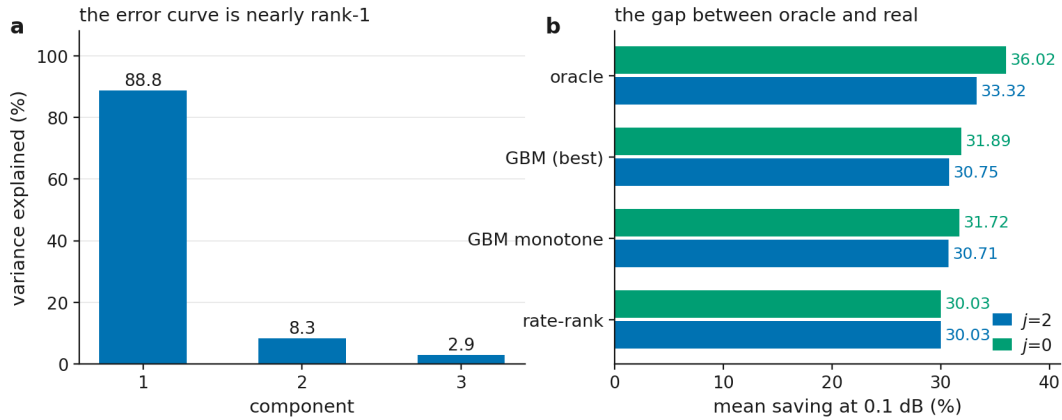
Treating the mask as purely spatial gives a pessimistic bound: a block-sparse kernel on $B\times B$ tiles that must run any tile containing one needed position keeps 37.4% of the ideal 38.6% at $B=4$ and collapses by $B=32$. That bound is far too pessimistic, because 99.708% of a DepthConvBlock is pointwise: $8C^2$ per position against $9C$ for the depthwise 3×3 at $C=384$. A 1×1 is a per-position matmul, so the kept positions can be gathered into a dense matrix at any granularity; charging only the depthwise term to the block-sparse bound leaves the realisable saving within 0.09 points of ideal even at $B=32$.



Sekil 9. Measured rather than argued: gather, two pointwise convolutions, scatter, at the shapes the decoder runs. Time tracks the fraction of positions kept to within 0.008–0.036, and the indices are a random permutation — the worst case for locality, where a real mask is contiguous.

9 What did not work

The routing axis is close to exhausted, and saying so is more useful than another table of small wins. The shipped rate-rank surrogate is rank-1: one learned scale per cell times one shape shared by all cells. On the test set that assumption is good to 91.7% of the variance in the log-error curve. An oracle that knows each cell's scale perfectly and shares one shape reaches 32.64% against the full oracle's 33.32%, so *everything* a richer model could add over rate-rank's functional form is worth 0.68 points — and 0.13 of that at the lowest rate.



Sekil 10. (a) Singular values of the centred log-error curves: the first component carries most of the structure, which is the assumption the shipped surrogate makes. (b) End-to-end, at 0.1 dB, scored by the allocation each router causes and with the decibel measured on the true error rather than on the router's own prediction.

Table 6. Seven attempts on the routing and allocation side, at 0.1 dB with 64 px cells. The compute axis moved by eight points; this axis moved by less than one.

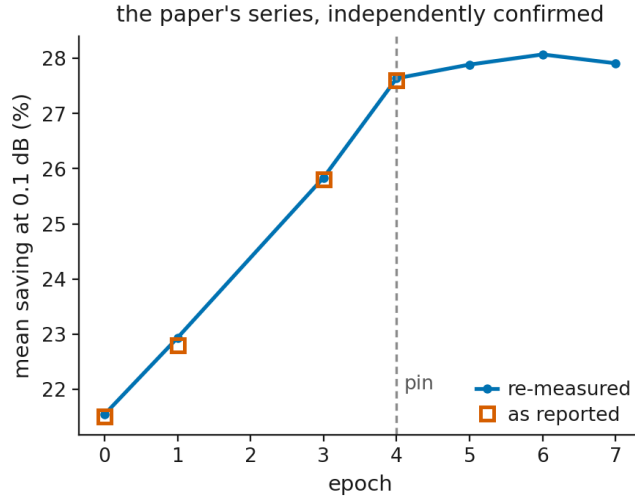
what was tried	result
gradient boosting, monotone increments	30.71% vs 30.03% for the rank-1 surrogate — +0.68
log-space bias correction (Duan smearing)	30.75% — +0.04 over the same model without it
MLP (128–64) on the same targets	30.07% — 0.68 <i>below</i> the trees
boosting each exit independently	27.00% — three points below rank-1: nothing stops it predicting a curve no ladder can produce
four stem statistics instead of all 26 inputs	30.68% vs 30.71% — the input axis is spent; dropping the neighbourhood group <i>helps</i>
smoothness-regularised allocation	halves the band (1.81→0.80 points at qp32) and loses slightly more than that in distortion
per-frame budget from predicted distortion	only 66–71% of frames stay under budget and the saving collapses; a per-frame lambda on the <i>true</i> decibel keeps 53/53, so the mechanism is fine and the predictions are not

10 The main experiment is unchanged, and re-verified

Nothing in this report modifies runs/RECIPE512 or any source file of the main experiment. As a side effect of re-measuring every epoch with today's code — the measurement pipeline was corrected twice this week, so the archived per-epoch files are not comparable to each other — the paper's own epoch series is reproduced independently.

Table 7. Mean saving at 0.1 dB over 53 CTC frames, re-measured with the current scripts. Epoch 2 has no checkpoint on disk. Beyond the pin the series flattens and then dips, and the ladder's spread stops improving, which is three independent reasons the pin is well placed.

epoch	as reported	re-measured	spread at qp32 (dB)
0	21.5	21.54	0.259
1	22.8	22.93	0.229
3	25.8	25.84	0.213
4 (pin)	27.6	27.64	0.199
5	—	27.89	0.198
6	—	28.07	0.198
7	—	27.91	0.197



Sekil 11. The re-measured series against the four values the paper reports. The pin is where the ladder's spread stops improving.

11 Limitations, and what is being trained now

The 36.02% is an oracle. It is the allocation a Lagrangian makes when it knows the true per-cell error, which is what the signalled configuration attains by transmitting the map, and it is the same quantity the main paper's headline reports. A trained decoder-side router reaches 31.89% at $j=0$ against 30.75% at $j=2$, so realisation falls from 92% to 88.5% when the two new rungs appear. That is expected and it is the point of the next paragraph: those rungs have never been optimised for the job.

The shallow rungs were never trained on a pixel loss. Under the main recipe the RD term is computed through `forward_random_depth`, which clamps, so a tile drawn at depth 0 or 1 is decoded at depth 2 and adapters 0 and 1 receive gradient only from the feature-space distillation term. They are therefore trained, but for imitation rather than for reconstruction. A fine-tune is running on a free card that changes exactly this and nothing else: no `--train_patched`, so the loss goes through `forward_all_exits`, which does not clamp; `--split_depth 0`; `--seam_repair none`; warm-started from the pinned checkpoint with the anchor weight unchanged so the deepest exit stays fixed to the release. If the shallow rungs improve, both the oracle and the router improve with them.

Other open items. The adapter cost for exits 0 and 1 is taken to be the same as exit 2's, which is what the shipped cost model implies but has not been re-derived for those exits. The head-term verification was run on $j=2$ allocations; $j=0$ maps have larger depth differences and therefore wider bands, so it should be repeated there. And the BD integration for $j=0$ has not been done — the tables above stop at the 0.1 dB operating point for that configuration.

What this report does not claim. It does not claim a better network, a better router, or a better training recipe. The network is the main experiment's, unmodified; the router is worse than an oracle by a margin this report measures rather than hides; and the only training change is the one described above, which is still running.

12 How to reach 36%: the three changes, concretely

Nothing below is a new module. Each item is a change to the decode path, and the three compose.

(1) Replace the tiled decode with a dilated one. Instead of cutting the frame and calling the decoder per tile, keep the frame whole. Compute $D = \text{dilate}(d)$ once, where the dilation is repeated 3×3 max-minus-one in blocks; then for each group g , run it where $D \geq g \cdot \text{bpe} + 1$ and leave the rest untouched. Two details are load-bearing and both were bugs the first time. The receptive field must be counted in *blocks*, not exits — an exit is two blocks, and counting in exits puts the deepest uniform decode 0.58 dB off. And each position's feature must be frozen at its own exit: the dilation keeps computing there to serve a deeper neighbour, so reading the end of the loop through the shallow adapter costs 0.015 dB, the same size as the effect being measured.

(2) Decide on 64 px cells instead of 256 px tiles. The allocation is a per-cell argmin of $m_k + \lambda c_k$ as before; only the grid changes. Charge the band by dilating the resulting map and counting blocks, not by a halo model. 32 px is measurably worse than 64 once the band is charged.

(3) Stop clamping the exit map. `forward()` applies `exit_map.clamp(min=j)`; with no cut, j has no function. Exits 0 and 1 already carry trained adapters. Take the argmin over all K exits using the unclamped cost vector — note that the shipped `exit_costs` already clamps its own cost entries, so a cost vector for $j=0$ has to be rebuilt or the shallow options are priced at a rate no decoder charges.

Tablo 8. The three changes, applied in order to the pinned epoch-4 checkpoint, oracle allocation. No weight is retrained at any step.

step	mean saving at 0.1 dB	delta
the paper today (tiled, $j=2$)	27.64%	—
+ per-position depth (256 px cells)	31.51%	+3.87
+ 64 px cells	33.32%	+1.81
+ unlocked shallow rungs ($j=0$)	36.02%	+2.70

13 Protocol and conventions

Three pairs of conventions are live in this codebase and each pair differs by more than the effects being reported, so every number above states which half it uses.

The decibel. Pooling every cell of every frame into one MSE is the natural form for the Lagrangian; averaging a per-frame decibel is what `test_video.py` does and therefore what a published DCVC-UF number means. On an identical allocation the two differ by 0.023–0.033 dB, a third of the budget, with pooling always the flattering one. Everything here is per-frame.

The saving. A saving can be measured against a released decode (denominator 1) or against our own deepest path (denominator 1.0095, which pays seam repair). Everything here is against the release. Mixing the two put the tiled frontier above the per-position ones at the highest rate the first time Figure 5 was drawn.

The budget. A budget met on average is not a budget met on every frame. One λ per rate lets a frame that gives up little subsidise one that gives up a lot; a per-frame λ forbids it and costs about 2.5 points. The main paper and this report both use one λ per rate, so the two are comparable; Section 9 reports what happens when the per-frame version is driven by predictions instead of by the truth.

The test set. 53 CTC frames, one frame per sequence, spanning UVG, MCL-JCV and the HEVC classes. Absolute quality is reported in DCVC-UF's own metric, $(6Y + U + V)/8$ computed in 4:2:0 on 0–255. The released decoder scores 32.536, 38.135 and 43.155 dB at qp 0, 32 and 63; the ladder's deepest exit reproduces it to 0.000, 0.011 and 0.028 dB.

12 Reproducing every number

Each figure and table above names the file it was read from. The measurement scripts live outside the main experiment, and the main experiment's source tree has zero modifications:

```
adaptive_depth.py – the per-position decode and its exactness check
granularity.py – the allocation-cell sweep with the band charged by dilation
head_term.py – idealised versus real decode, all 53 frames
frontier.py, frontier_deployed.py – the swept frontiers
router_data.py, router_fit.py – per-cell features and the router comparison
smooth_alloc.py, mask_structure.py, gather_bench.py – the three negative
and implementability results
epoch_psnr.py – per-epoch PSNR in the codec's own 6:1:1 4:2:0 metric
```

References

- [1] Zhang et al. Be Your Own Teacher: Improve the Performance of Convolutional Neural Networks via Self Distillation. ICCV 2019.
- [2] Mao et al. AdaRevD: Adaptive Patch Exiting Reversible Decoder Pushes the Limit of Image Deblurring. CVPR 2024.
- [3] Adaptive Patch Exiting for Scalable Single Image Super-Resolution. ECCV 2022.
- [4] Liu et al. Deep Adaptive Inference Networks for Single Image Super-Resolution (AdaDSR). ECCV 2020.
- [5] Chu et al. Improving Image Restoration by Revisiting Global Information Aggregation (TLC). ECCV 2022.
- [6] Kong et al. ClassSR: A General Framework to Accelerate Super-Resolution Networks by Data Characteristic. CVPR 2021.
- [7] Multi-exit self-distillation with appropriate teachers. FITEE 2024.
- [8] Bjontegaard. Calculation of Average PSNR Differences between RD-Curves. VCEG-M33, 2001.